COP vs CP

The solution of the problem and the optimisation of its objective functions are two completely independent tasks, only connected by their shared parameter n. This is because we know a-priori that the schedule will contain all possible matches (as 2 elements subsets of the set of all teams) and the home or away state of a match does not interact with when it will occur (as it would have been, say, if we imposed that no team plays more than k matches in a row at home). This also means that, with a fixed n, once that an optimally balanced home/away match spread is obtained, it can be overlayed on any solution with no additional computation, making it optimal. Conversely, the construction of the problem solution has no bearing on optimality and all solutions have the chance to become optimal.

Decision Variables:

match_vars[w, p] two matrices of natural numbers from 1 to n^2-n. The function match_number gives each match a unique ID. These are layed out in the grid representing weeks and periods.

x[w,p] and y[w,p] two matrices of variable Matches that indicate which team plays in each weep and period. They don’t encode the home and away team, but simply are such that $\forall w \in W, p \in P, x[w,p] < y[w,p]$

Swap, an array of variable bool that, for each match ID, dictates whether the order home-away dictated by lexicographical ordering has to be reversed. Used exclusively in the COP section.

Home count, an array of variable int that, for each team, counts how many home games they play.

Imbalance, an array of variable int that, for each team, counts how far from optimal their home/away balance is.

Constraints:

1) every team plays with every other team exactly once
2) every team plays once a week
3) every team plays at most twice in the same period over the tournament.

Result: Every solution has a symmetry group that is isomorphic to $S_{n/2} \times S_{n-1} \times S_n$,

Every problem solution is invariant through three different operations: the permutation of its rows, the permutation of its columns and the permutation of its team labels.
We can express each of these operation types with the group of permutations (Symmetric Group) of a set of size, respectively, n//2, n-1 and n.
All of these operations commute with each other, in a group theory sense, but not with themselves. Since rows and column permutations can be expressed via, respectively, right

and left matrix multiplication with elemental swap matrices, this follows naturally from the associativity of matrix products. Then, since all permutations can be decomposed into compositions of transpositions, the fact that label permuting and row/column permuting commute can be proved manually for simple transpositions and it will ripple to every element of their respective group.
Indicating with S_k the Symmetric Group of order k, this makes it so that the group of invariance operations for a single solution is isomorphic to $S_{n/2} \times S_{n-1} \times S_n$, the direct product of the groups responsible for each permutation type.
The group has cardinality n!*(n/2)!*(n-1)!. This already large number would then have to be multiplied by 2^(n^2-n) if we hadn’t split the solution and optimization components of problem solving (if we considered each match as an ordered pair rather than a two element set).
This means, in the space of all possible team assignment of cardinality n^2(n^2-n), that each solution class has n!*(n/2)!*(n-1)! representatives: the goal of our symmetry breaking will be to elect a smaller subset of canonical representatives. This will cut down our search space immensely, by only considering these instead of all possible matrices.

We will do it constructively, by imposing no further constraints on a generic solution matrix. But we’ll need an introductory lemma.

Lemma: There exists a function from teams to periods such that all periods has exactly two teams mapped to it. This represents which period the input team will play only once.

Given n-1 weeks, each period will see n-1 different matches, so 2n-2 team instances. Trivially, the constraint n.3 can only be satisfied with at maximum 2n team instances per period. So, out of this complete collection, only two team instances are missing. They can’t be by the same team, because it would mean that that team would have to play n-1 matches in (n/2 - 1) periods, which without break either the constraint n.2 or n.3, since 2*(n/2 - 1) = n-2 < n-1 . This means that each period possesses exactly two teams, thus forth called defective, that only play once during it. Conversely, the constraint n.2 makes it so that each team has only one period it can be defective in.

Result: From the n!*(n/2)!*(n-1)! different representations of a solution matrix it is possible to extract 2^(n/2)*(n/2)! representatives.

[Imagine that in the beginning the teams are each represented by a different symbol, with no order. Team labelling is assigning a total order to the symbols, that is being able to map them natural numbers onto them monotonically.]
Out of all teams, we designate one the have the label 1. This implies n possible choices. Using the lemma, Team 1 will have exactly one period in which it is defective, and within this exactly one match in which it plays. Singling out the week in which this occurs, we want to assign all other labels such that team 2k and 2k+1 play each other during that week. Team 1 will play team 2, and we want to assign a team 3 such that it will play team 4 and so on. This implies (n/2)! choices in mapping each match happening that week with a different value of k and another 2^(n/2) in choosing which of the two teams playing each other in each match gets the slightly bigger label. Once this is done, there is exactly one way to order the rows such that in the week we’re examining they appear as

[1,2
3,4
…]
Once the rows are ordered, we can similarly induce a total order in the columns. The columns in which the teams 1 plays the earliest will come before the ones in which it plays later. Ties will be resolved with the order of the team 1 is against.
Thus, we can reduce any solution matrix in canonical form via only 2^(n/2)*(n/2)! choices.